

Technical Architecture Document

SaaS architecture, database schema, project structure, and configuration

Prepared from the Stratedge codebase

Last updated: 2026-06-13

Edgecipline Technical Architecture Document

Last updated: 2026-06-13

1. Architecture Summary

Edgecipline is a multi-platform SaaS product for retail traders. It combines an AI-assisted trade journal, screenshot-to-trade extraction, performance analytics, discipline tracking, psychology insights, weekly reports, subscriptions, notifications, and an internal admin console.

The recommended architecture is a modular full-stack SaaS:

- Static-export Next.js frontend for web and mobile shell delivery.
- Capacitor Android wrapper for mobile distribution.
- Node.js/Express API for business logic and integrations.
- MongoDB for flexible product data and analytics-friendly document storage.
- Redis + BullMQ for asynchronous OCR/AI processing.
- Cloudinary for uploaded screenshot storage.
- Google Vision / Tesseract / Gemini for OCR and AI extraction.
- Firebase Authentication for Google sign-in and mobile-friendly auth.
- JWT + rotating refresh tokens for backend sessions.
- Razorpay for India-first subscription payments.
- Sentry + Winston logs for production observability.

At scale, the system should be deployed as three independently scalable runtime units:

1. Frontend static app served by CDN.
2. API server handling auth, CRUD, analytics, payments, admin, and orchestration.
3. OCR worker fleet processing image extraction jobs from Redis/BullMQ.

2. Core Product Domains

User Domain

Handles account creation, login, Google auth, terms acceptance, subscription state, onboarding state, token revocation, and password reset.

Trade Journal Domain

Handles Forex and Indian market trade creation, editing, deletion, screenshot references, manual entry, psychology metadata, setup checklist outcomes, and soft deletion.

OCR / AI Extraction Domain

Handles screenshot upload, Cloudinary persistence, async job creation, OCR text extraction, AI extraction, validation, confidence scoring, non-trade rejection, and draft confirmation.

Analytics Domain

Computes dashboard metrics, equity curve, performance by strategy/session/instrument, psychology cost, self-awareness, trading DNA, repeated mistakes, drawdown, and pattern detection.

Setup And Discipline Domain

Handles setup strategies, rule checklists, per-session checklist tracking, setup score, A+ classification, checklist reminders, and plan adherence analytics.

Report Domain

Generates rolling weekly reports from trade history and optional AI feedback.

Subscription Domain

Handles free upload gating, Razorpay order creation, payment verification, subscription activation, expiry, and admin overrides.

Notification Domain

Handles device tokens, notification preferences, in-app notification history, push delivery status, morning mentor reminders, weekly report reminders, and checklist reminders.

Admin Domain

Provides separate admin authentication and operations for users, payments, trades, feedback, notifications, and platform analytics.

3. Recommended Tech Stack

Frontend

Layer Choice Reason

--- --- ---

Web framework Next.js App Router with static export Good developer experience, route-based product organization, CDN-friendly static deployment, works with Capacitor.

Language TypeScript-first JavaScript Type safety where available while preserving current JS modules.

UI React 19 Mature ecosystem, component reuse across web/PWA/mobile shell.

Styling Tailwind CSS v4 plus local component styles Fast iteration for early-stage SaaS, easy responsive layouts.

Server state TanStack React Query Caching, loading states, retries, invalidation after trade writes.

HTTP client Axios + axios-retry Centralized API client with retry semantics.

Charts Recharts Sufficient for dashboards, P&L curves, performance cards.

Icons lucide-react Consistent UI icon system.

Mobile shell Capacitor Android Lets the same static app ship as Android APK.

Error tracking Sentry for Next.js Client-side production error visibility.

Recommendation: keep Next.js as a static export while the app is API-driven. If SSR becomes necessary for SEO-heavy public pages, split marketing/public pages from the authenticated app shell.

Backend

Layer Choice Reason

--- --- ---

Runtime Node.js Good fit for I/O-heavy SaaS and AI/API orchestration.

API framework Express 5 Simple, mature, already in codebase.

Database ORM Mongoose Flexible schema modelling for evolving analytics/trade objects.

Auth Backend JWT + rotating refresh tokens + Firebase Admin for Google tokens Keeps backend authorization authoritative while allowing Google/mobile auth.

Queue BullMQ Reliable Redis-backed background processing for OCR/AI work.

Cache Redis Response caching, queue backend, invalidation after writes.

File upload Multer + file-type validation Standard multipart upload flow with validation.

Image storage Cloudinary Managed image hosting, public IDs, deletion support.

OCR Google Cloud Vision primary, Tesseract fallback Strong broker screenshot OCR, local fallback for resilience.

AI extraction Gemini 2.5 Flash primary, text fallback Cost-effective multimodal extraction for screenshots.

Payments Razorpay Best fit for India-first subscription payments.

Email Resend and/or SMTP/Nodemailer Transactional emails, OTP, reports, reminders.

Logging Winston + Morgan Structured application logs and request logs.

Error tracking Sentry Node Crash/error visibility with header/cookie scrubbing.

Security Helmet, CORS allowlist, rate limiting, input sanitization, central error handler Baseline SaaS hardening.

Data Storage

Store Usage

--- ---

MongoDB Users, trades, Indian trades, OCR jobs, reports, payments, notifications, feedback, setup strategies.

Redis BullMQ jobs, short-lived cache, operational queue state.

Cloudinary Uploaded screenshots and setup reference images.

Deployment

Recommended deployment topology:

- CDN/static host for frontend/out.
- API server as stratedge-api.
- OCR worker as stratedge-ocr-worker.
- MongoDB Atlas for database.
- Managed Redis for queue/cache.
- PM2 or container orchestration for backend processes.
- Nginx/Cloudflare in front of API for TLS, compression, and static security headers.

4. High-Level System Flow

Authentication Flow

1. User registers with email/password or signs in with Google.
2. Google login sends Firebase ID token to backend.
3. Backend verifies credentials or Firebase token.
4. Backend creates or finds User.
5. Backend issues short-lived JWT and refresh token.
6. Protected requests send Authorization: Bearer <token>.
7. Auth middleware verifies JWT, checks user, checks token version, and attaches req.user.

Screenshot Upload Flow

1. Client uploads screenshot to /api/upload/trade.
2. API validates file type, size, user subscription, market type, broker, and trade subtype.
3. API uploads image to Cloudinary.
4. API creates an OCRJob record.
5. API enqueues BullMQ job.
6. Worker extracts OCR text using Google Vision or Tesseract.
7. Worker runs Gemini Vision extraction, with text fallback if needed.
8. Worker validates parsed trade data and confidence.
9. Worker stores extracted draft data in OCRJob.
10. Client polls job/status endpoint.
11. User reviews and confirms extracted trade.
12. Confirmed trade is saved into Trade or IndianTrade.
13. User-specific cache is invalidated.

Analytics Flow

1. Client requests dashboard or analytics endpoint.
2. API reads user trade data filtered by market and soft-delete state.
3. Analytics utilities compute derived stats.
4. Expensive aggregate responses may be cached briefly in Redis.
5. Any trade write clears the user cache.

Payment Flow

1. User selects paid plan.
2. API creates Razorpay order and Payment record.
3. Client opens Razorpay checkout.
4. Client sends payment ID/signature to backend.
5. Backend verifies signature.
6. Backend marks payment completed and updates User.subscriptionStatus, subscriptionPlan, and subscriptionExpiry.

5. Recommended Project Structure

The current repo already follows this structure. Keep it, but standardize naming and ownership boundaries.

- stratedge/
- backend/
- admin/
- controllers/ Admin business handlers
- routes/ /api/admin/* route definitions
- config/
- index.js Central environment/config loader
- db.js MongoDB connection
- redis.js Redis/BullMQ connection
- cloudinary.js Cloudinary client
- firebaseAdmin.js Firebase Admin SDK
- constants/
- terms.js Terms/privacy version constants
- controllers/
- authController.js
- uploadController.js
- tradeController.js
- indianTradeController.js
- analyticsController.js
- indianAnalyticsController.js
- dashboardController.js
- setupController.js
- checklistController.js
- weeklyReportController.js
- paymentController.js
- notificationController.js
- feedbackController.js
- profileController.js
- aiCoachController.js
- disciplineController.js
- patternController.js
- timelineController.js
- jobs/
- dataCleanupCron.js
- weeklyReportsCron.js
- morningMentorCron.js
- middleware/
- adminAuth.js
- authMiddleware.js
- cacheMiddleware.js
- errorHandler.js
- rateLimiter.js
- sanitizeInput.js
- timeout.js
- uploadMiddleware.js
- validateObjectId.js
- validateNumbers.js
- models/
- Users.js
- RefreshToken.js
- Trade.js
- IndianTrade.js
- OCRJob.js
- ExtractionLog.js
- SetupStrategy.js

ChecklistTracking.js
WeeklyReport.js
Payment.js
Notification.js
NotificationHistory.js
NotificationPreference.js
DeviceToken.js
ChecklistNotificationSetting.js
Feedback.js
queues/
ocrQueue.js
repositories/
user.repository.js
trade.repository.js
indianTrade.repository.js
setup.repository.js
weeklyReport.repository.js
routes/
authRoutes.js
uploadRoutes.js
tradeRoutes.js
tradeStatusRoutes.js
indianMarketRoutes.js
dashboardRoutes.js
analyticsRoutes.js
indianAnalyticsRoutes.js
setupRoutes.js
checklistRoutes.js
weeklyReportRoutes.js
paymentRoutes.js
notificationRoutes.js
profileRoutes.js
feedbackRoutes.js
services/
tokenService.js
upload.service.js
trade.service.js
tradeLifecycle.service.js
tradeProcessingService.js
ocrJob.service.js
visionOcrService.js
ocrService.js
aiExtractionService.js
geminiService.js
parsingService.js
extractionQualityService.js
analyticsSnapshotService.js
weeklyReport.service.js
notificationService.js
morningMentorService.js
mailService.js
utils/
ApiError.js
asyncHandler.js
cache.js
cacheUtils.js
dateUtils.js
metricEngine.js
disciplineAnalytics.js
aiCoachFeed.js
patternDetection.js
psychologyCost.js

psychologyTimeline.js
tradingDNA.js
tradeEvaluation.js
logger.js
workers/
ocrWorker.js
__tests__/
unit/
integration/
server.js
package.json
frontend/
app/
layout.tsx
providers.tsx
page.tsx
login/
register/
forgot-password/
reset-password/
verify-otp/
accept-terms/
dashboard/
upload-trade/
add-trade/
trades/
indian-market/
dashboard/
upload-trade/
add-trade/
trades/
analytics/
discipline/
setups/
analytics/
page.js
ai-coach/
ai-insights/
discipline/
mistakes/
patterns/
psychology/
psychology-cost/
self-awareness/
trading-dna/
checklist/
setups/
weekly-reports/
intelligence/
trading-dna/
discipline/
psychology-timeline/
notifications/
profile/
support/
admin/
privacy-policy/
terms/
components/
shared cross-feature components
config/
api.js

seo.ts
marketThemes.js
context/
MarketContext.js
features/
auth/
api/
components/
hooks/
trade/
api/
components/
hooks/
analytics/
api/
components/
hooks/
dashboard/
api/
components/
hooks/
ai-coach/
shared/
components/
hooks/
plugins/
Capacitor plugins
public/
logos, sample screenshots, manifest
services/
api clients and notification services
utils/
auth.js
metricEngine.js
queryInvalidation.js
marketNavigation.js
imageUpload.js
currencyFormatter.js
android/
Capacitor Android project
package.json
docs/
product-requirements-document.md
technical-architecture-document.md
scripts/
security-scan.js
check-mobile-auth-storage.js
check-mojibake.js

6. Database Schema

This project uses MongoDB collections via Mongoose models. The word "table" below maps to a MongoDB collection.

Relationship Overview

- User owns most user-scoped records.
- Trade.user, IndianTrade.user, SetupStrategy.user, ChecklistTracking.user, WeeklyReport.user, Payment.user, Feedback.user, DeviceToken.user, and notification records reference User.
- RefreshToken.userId references User for backend session rotation.
- ChecklistNotificationSetting.strategyId optionally references SetupStrategy.
- OCRJob.confirmedTradeId points to a confirmed Trade or IndianTrade, with confirmedTradeCollection identifying the collection.
- Trade.deletedBy and IndianTrade.deletedBy reference User for soft delete audit.

User

Collection: users

Purpose: account, auth, role, subscription, onboarding, and security state.

Fields:

- name: user display name.
- email: unique lowercase email.
- password: hashed local password, excluded by default.
- authProvider: local or google.
- googleId: Google account ID, unique when present.
- role: user or admin.
- avatar: profile image URL.
- resetPasswordOTP: OTP used for password reset.
- resetPasswordOTPExpires: OTP expiry.
- resetPasswordToken: short-lived token issued after OTP verification.
- resetPasswordTokenExpires: reset token expiry.
- otpAttempts: OTP attempt counter.
- otpLockUntil: temporary OTP lock time.
- subscriptionStatus: inactive, active, or expired.
- subscriptionPlan: free, monthly, or yearly.
- subscriptionExpiry: subscription end date.
- totalPaid: total lifetime amount paid.
- lastLogin: last login timestamp.
- freeUploadUsed: whether free screenshot upload has been consumed.
- hasSeenWelcomeGuide: dashboard onboarding flag.
- isOnboardingCompleted: broader onboarding completion flag.
- termsAcceptance.acceptedTerms: terms accepted boolean.
- termsAcceptance.acceptedPrivacy: privacy policy accepted boolean.
- termsAcceptance.acceptedAt: acceptance timestamp.
- termsAcceptance.termsVersion: accepted terms version.

- tokenVersion: increments to invalidate existing JWTs.
- loginAttempts: login rate/security counter, excluded by default.
- loginLockedUntil: temporary login lock time, excluded by default.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique email.
- Sparse unique googleId.
- role + subscriptionStatus + subscriptionExpiry.
- subscriptionStatus + subscriptionExpiry.

RefreshToken

Collection: refreshtokens

Purpose: stores hashed rotating refresh tokens for secure session renewal and replay detection.

Fields:

- userId: reference to User.
- tokenHash: SHA-256 hash of raw refresh token, unique.
- family: rotation family ID; replay can revoke the whole family.
- deviceInfo.userAgent: client user agent.
- deviceInfo.ip: client IP.
- expiresAt: token expiry.
- revokedAt: timestamp when rotated/revoked.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique tokenHash.
- family.
- TTL on expiresAt with 24-hour grace.
- userId + revokedAt + expiresAt.

Trade

Collection: trades

Purpose: Forex trades, and legacy/pending extraction fields for general trade upload processing.

Fields:

- user: reference to User.
- pair: forex pair or instrument label.
- type: BUY or SELL.
- quantity: quantity/position amount.
- lotSize: lot size.
- entryPrice: entry price.
- exitPrice: exit price.
- stopLoss: stop loss.
- takeProfit: take profit.
- profit: realized P&L.

- commission: broker commission.
- swap: swap/overnight fee.
- balance: account balance from screenshot if available.
- strategy: strategy/setup label.
- session: trading session.
- tradeDate: actual trade date.
- notes: user notes.
- riskRewardRatio: enum such as 1:1, 1:2, 1:3, 1:4, 1:5, custom.
- riskRewardCustom: custom R:R text.
- screenshot: screenshot URL shown in journal.
- imageUrl: original Cloudinary URL.
- marketType: defaults to Forex.
- tradeSubType: OPTION, EQUITY, or null for cross-market upload compatibility.
- broker: broker label.
- segment: market segment.
- instrumentType: instrument type.
- strikePrice: option strike when relevant.
- expiryDate: option expiry text when relevant.
- extractedText: cleaned OCR text.
- rawOCRText: raw OCR output.
- aiRawResponse: raw AI response.
- parsedData: mixed parsed extraction payload.
- extractionConfidence: 0-100 confidence score.
- isValid: whether extraction/trade passed validation.
- needsReview: whether user should verify fields.
- status: pending, processing, completed, or failed.
- ocrJobId: queue/job identifier.
- ocrJobName: job name.
- ocrAttempts: processing attempts count.
- queuedAt: when job was queued.
- processingStartedAt: when processing began.
- error: failure message.
- processedAt: completion timestamp.
- setupRules: array of { label, followed }, max 20.
- setupScore: percentage of setup rules followed.
- entryBasis: Plan, Emotion, Impulsive, Custom, or empty.
- entryBasisCustom: custom entry basis text.
- mood: 1-5 psychology state.
- confidence: Low, Medium, High, Overconfident, or empty.
- emotionalTags: up to 10 tags.
- mistakeTag: mistake label.
- lesson: post-trade lesson.
- wouldRetake: Yes, No, or empty.
- tradeQuality: Great, Average, Poor, or empty.
- deletedAt: soft delete timestamp.
- deletedBy: user/admin who deleted.
- deleteReason: audit reason.
- deletedSource: deletion source.
- createdAt, updatedAt: timestamps.

Important indexes:

- User/time indexes for journal queries.
- User/market/status indexes for upload/status flows.
- User/deletedAt/tradeDate/profit indexes for analytics.
- User/setupScore, mistakeTag, entryBasis indexes for discipline analytics.
- Sparse deletedAt.

IndianTrade

Collection: indiantrades

Purpose: Indian market trades for F&O options and intraday equities.

Fields:

- user: reference to User.
- pair: display symbol such as NIFTY 26000 CE.
- underlying: NIFTY, BANKNIFTY, stock ticker, etc.
- type: BUY or SELL.
- optionType: CE or PE.
- entryPrice: entry premium/price.
- exitPrice: exit premium/price.
- stopLoss: stop loss.
- takeProfit: target.
- profit: realized P&L in INR.
- strategy: strategy/setup label.
- session: trading session.
- tradeDate: trade date.
- notes: user notes.
- riskRewardRatio: enum such as 1:1, 1:1.5, 1:2, custom.
- riskRewardCustom: custom R:R text.
- screenshot: screenshot URL.
- segment: F&O, EQUITY, or empty.
- instrumentType: OPTION, EQUITY, or empty.
- stockSymbol: equity ticker.
- exchange: NSE, BSE, or empty.
- sharesQty: equity share quantity.
- sector: equity sector.
- strikePrice: option strike.
- expiryDate: expiry date.
- quantity: lots or quantity.
- lotSize: lot size.
- tradeType: INTRADAY, DELIVERY, SWING, or empty.
- brokerage: brokerage amount.
- sttTaxes: taxes/fees.
- entryBasis: plan/emotion/impulsive/custom state.
- entryBasisCustom: custom entry basis.
- setup: setup label.
- mistakeTag: mistake label.
- lesson: user lesson.

- setupRules: array of checklist rules.
- setupScore: setup adherence percentage.
- mood: 1-5 psychology state.
- confidence: confidence label.
- emotionalTags: up to 10 emotion tags.
- wouldRetake: Yes, No, or empty.
- tradeQuality: Great, Average, Poor, or empty.
- deletedAt, deletedBy, deleteReason, deletedSource: soft delete audit fields.
- createdAt, updatedAt: timestamps.

Important indexes:

- User/instrument/time indexes for market-specific journal views.
- User/deletedAt/time indexes for analytics.
- User/setupScore, mistakeTag, entryBasis indexes for behavioral analytics.

OCRJob

Collection: ocrjobs

Purpose: durable state for screenshot extraction jobs and reviewable extracted drafts.

Fields:

- user: reference to User.
- marketType: Forex or Indian_Market.
- tradeSubType: OPTION, EQUITY, or empty.
- broker: broker label.
- uploadedImage.imageUrl: Cloudinary URL.
- uploadedImage.publicId: Cloudinary public ID.
- uploadedImage.originalName: original file name.
- uploadedImage.mimeType: uploaded MIME type.
- uploadedImage.bytes: file size.
- requestedTradeDate: user-selected date before extraction.
- status: PENDING, PROCESSING, COMPLETED, FAILED, CANCELLED, or CONFIRMED.
- queueJobId: BullMQ job ID.
- queueJobName: queue job name.
- attemptsMade: retry count.
- processingStartedAt: processing start time.
- processedAt: processing end time.
- cancelledAt: cancellation time.
- confirmedAt: user confirmation time.
- confirmedTradeId: confirmed trade document ID.
- confirmedTradeCollection: forex, indian, or empty.
- error: failure message.
- extractedData: extracted draft payload.
- extractionConfidence: 0-100 confidence.
- legacyDraftFailureRetryCount: migration/legacy retry state.
- expiresAt: TTL expiry for unconfirmed jobs.
- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.

- user + status + createdAt.
- TTL on expiresAt.

ExtractionLog

Collection: extractionlogs

Purpose: audit log for OCR/AI extraction attempts.

Fields:

- user: reference to User.
- imageUrl: processed image URL.
- marketType: market context.
- extractedText: OCR text.
- parsedData: parsed payload.
- isSuccess: extraction success flag.
- aiUsed: whether AI was used.
- errorMessage: failure reason.
- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.
- isSuccess + createdAt.

SetupStrategy

Collection: setupstrategies

Purpose: user-defined trading setups and rule checklists.

Fields:

- user: reference to User.
- marketType: Forex or Indian_Market.
- name: setup name.
- referenceImages: array of { url, publicId }.
- rules: array of { label }.
- createdAt, updatedAt: timestamps.

Important indexes:

- user + marketType + createdAt.
- Unique user + marketType + name.

ChecklistTracking

Collection: checklisttrackings

Purpose: history of pre-trade checklist completions.

Fields:

- user: reference to User.
- market: Forex or Indian_Market.
- strategyName: setup/checklist strategy name.
- totalRules: total checklist rule count.

- followedRules: number followed.
- score: adherence score.
- isAPlus: whether checklist qualifies as A+.
- setupSimilarity: yes, partly, no, or empty.
- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.
- user + market + createdAt.

WeeklyReport

Collection: weeklyreports

Purpose: stores generated weekly/rolling reports and optional AI feedback.

Fields:

- user: reference to User.
- marketType: market context.
- periodType: currently rolling7d.
- weekStart: period start.
- weekEnd: period end.
- snapshot: generated report stats.
- aiFeedback: AI report commentary payload.
- aiModel: AI model used.
- promptVersion: report prompt version.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique user + marketType + periodType + weekStart + weekEnd.
- user + marketType + weekStart.
- user + marketType + periodType + createdAt.

Payment

Collection: payments

Purpose: payment and subscription transaction audit.

Fields:

- user: reference to User.
- amount: amount paid.
- currency: defaults to INR.
- status: pending, completed, refunded, or failed.
- paymentMethod: razorpay, manual, or stripe.
- transactionId: unique internal/external transaction ID.
- planType: 3_months or custom.
- expiryDate: resulting subscription expiry.
- notes: admin/payment notes.
- razorpayOrderId: Razorpay order ID.
- razorpayPaymentId: Razorpay payment ID.
- razorpaySignature: verification signature.

- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.
- status + createdAt.
- Sparse razorpayOrderId.
- Sparse unique razorpayPaymentId for Razorpay payments.

Notification

Collection: notifications

Purpose: legacy/simple notification records.

Fields:

- title: notification title.
- message: body copy.
- type: payment, feedback, or system.
- isRead: read flag.
- userId: reference to User.
- metadata: arbitrary payload.
- createdAt, updatedAt: timestamps.

Important indexes:

- userId + createdAt.
- userId + isRead.

NotificationHistory

Collection: notificationhistories

Purpose: rich notification delivery log and in-app notification feed.

Fields:

- user: reference to User.
- type: notification category such as revenge trading, overtrading, morning mentor, weekly insight, payment, feedback, system.
- title: title.
- body: message body.
- data: arbitrary structured payload.
- deepLink: in-app target route.
- sourceType: trade, weekly_report, cron, or system.
- sourceId: related object ID.
- dedupeKey: unique user-scoped key preventing duplicate notifications.
- status: created, sent, failed, partial, or skipped.
- isRead: read state.
- readAt: read timestamp.
- sentAt: send timestamp.
- delivery.successCount: successful token deliveries.
- delivery.failureCount: failed token deliveries.
- delivery.invalidTokens: invalid token list, capped at 100.
- delivery.error: delivery error text.

- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.
- user + isRead + createdAt.
- Unique user + dedupeKey.

NotificationPreference

Collection: notificationpreferences

Purpose: per-user notification settings.

Fields:

- user: unique reference to User.
- pushEnabled: push toggle.
- inAppEnabled: in-app toggle.
- smartCoach: smart coaching toggle.
- revengeTrading: revenge alert toggle.
- overtrading: overtrading alert toggle.
- setupDiscipline: setup discipline alert toggle.
- repeatedMistakes: repeated mistakes alert toggle.
- moodRisk: mood risk alert toggle.
- noStopLoss: no-stop alert toggle.
- weeklyInsight: weekly insight alert toggle.
- morningMentor: morning mentor toggle.
- quietHours.enabled: quiet hours toggle.
- quietHours.start: quiet start time.
- quietHours.end: quiet end time.
- quietHours.timezone: timezone.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique user.

DeviceToken

Collection: devicetokens

Purpose: stores push notification tokens per device.

Fields:

- user: reference to User.
- token: unique FCM/device token.
- platform: android, ios, or web.
- deviceId: client device ID.
- appVersion: app version.
- enabled: whether token is active.
- lastSeenAt: last token heartbeat.
- failureCount: delivery failure counter.
- revokedAt: token revocation time.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique token.
- user + enabled.
- lastSeenAt.
- TTL on lastSeenAt after 90 days.

ChecklistNotificationSetting

Collection: checklistnotificationsettings

Purpose: stores pre-trade checklist reminder settings.

Fields:

- user: reference to User.
- enabled: reminder toggle.
- strategyId: optional reference to SetupStrategy.
- strategyName: denormalized strategy name.
- market: Forex or Indian_Market.
- notificationTime: local reminder time.
- repeatMode: daily, weekdays, or custom.
- customDays: 1-7 day numbers.
- persistent: whether notification persists.
- resetEnabled: whether checklist resets.
- resetTime: reset time.
- createdAt, updatedAt: timestamps.

Important indexes:

- Unique user + market.

Feedback

Collection: feedbacks

Purpose: user-submitted support, bug, and feature feedback.

Fields:

- user: reference to User.
- type: BUG, FEATURE_REQUEST, or GENERAL_FEEDBACK.
- subject: subject line.
- message: feedback body.
- status: PENDING, IN_PROGRESS, or RESOLVED.
- adminNotes: internal notes.
- screenshot: optional screenshot URL.
- createdAt, updatedAt: timestamps.

Important indexes:

- user + createdAt.
- status + createdAt.

7. API Surface

Public/User API

- /api/auth: register, login, Google login, refresh, logout, forgot password, verify OTP, reset password, accept terms, profile.
- /api/upload: screenshot upload and OCR job lifecycle.
- /api/trades: Forex trade CRUD.
- /api/trade: OCR/trade status polling.
- /api/indian/trades: Indian market trade CRUD.
- /api/dashboard: dashboard snapshot and summary.
- /api/analytics: Forex analytics.
- /api/indian/analytics: Indian analytics.
- /api/setups: setup strategy CRUD.
- /api/checklists: checklist tracking.
- /api/reports: weekly reports.
- /api/payments: Razorpay orders and verification.
- /api/notifications: notification feed/preferences.
- /api/profile: device token/profile updates.
- /api/feedback: user feedback.

Admin API

- /api/admin/auth: admin login.
- /api/admin/users: user and subscription management.
- /api/admin/payments: payment management.
- /api/admin/trades: trade review/management.
- /api/admin/analytics: platform analytics.
- /api/admin/notifications: send/manage notifications.
- /api/admin/feedback: feedback review.

8. Configuration And Environment Variables

Required Backend Variables

- NODE_ENV: development, test, or production.
- PORT: backend port, default 5000.
- MONGO_URI: MongoDB connection string.
- JWT_SECRET: user JWT secret, minimum 32 characters.
- ADMIN_JWT_SECRET: admin JWT secret, minimum 32 characters and different from JWT_SECRET.
- CLOUD_NAME: Cloudinary cloud name.
- CLOUD_API_KEY: Cloudinary API key.
- CLOUD_API_SECRET: Cloudinary API secret.

Strongly Recommended Backend Variables

- REDIS_URL: Redis connection string for BullMQ/cache.
- GEMINI_API_KEY: Gemini API key for AI extraction.
- GEMINI_MODEL: default Gemini model, currently gemini-2.5-flash.
- GEMINI_TRADE_MODEL: override model for trade extraction.
- GOOGLE_VISION_PROJECT_ID: Google Vision project ID.
- GOOGLE_VISION_CLIENT_EMAIL: Vision service account email.
- GOOGLE_VISION_PRIVATE_KEY: Vision service account private key.
- FIREBASE_PROJECT_ID: Firebase Admin project ID.
- FIREBASE_CLIENT_EMAIL: Firebase Admin service account email.
- FIREBASE_PRIVATE_KEY: Firebase Admin private key.
- RAZORPAY_KEY_ID: Razorpay public key ID.
- RAZORPAY_KEY_SECRET: Razorpay secret.
- SENTRY_DSN: backend Sentry DSN.
- ALLOWED_ORIGINS: comma-separated allowed frontend origins.
- ALLOWED_VERCEL_PREVIEWS: explicit preview URLs for production.

Optional Backend Variables

- LOG_LEVEL: logger verbosity.
- MONGO_DNS_SERVERS: DNS servers for Mongo lookup.
- MONGO_MAX_POOL_SIZE: Mongo pool max, default 50.
- MONGO_MIN_POOL_SIZE: Mongo pool min, default 5.
- OCR_QUEUE_NAME: queue name.
- OCR_JOB_ATTEMPTS: OCR job retry count.
- OCR_JOB_BACKOFF_MS: OCR retry backoff.
- OCR_JOB_INITIAL_DELAY_MS: initial processing delay.
- OCR_WORKER_CONCURRENCY: worker concurrency, default 5.
- OCR_WORKER_LOCK_DURATION_MS: BullMQ lock duration.
- UPLOAD_MAX_FILE_SIZE_BYTES: upload size limit, default 2 MB.
- ENABLE_DATA_CLEANUP_CRON: enable OCR data cleanup.
- DATA_CLEANUP_CRON_SCHEDULE: cleanup schedule.
- CLEANUP_RAW_OCR_DAYS: raw OCR retention.

- CLEANUP_AI_RESPONSE_DAYS: AI response retention.
- CLEANUP_IMAGES_DAYS: image cleanup retention.
- CLEANUP_BATCH_SIZE: cleanup batch size.
- ENABLE_WEEKLY_REPORTS_CRON: weekly report reminder cron.
- WEEKLY_REPORTS_CRON: weekly report cron schedule.
- ENABLE_MORNING_MENTOR_CRON: morning mentor cron.
- MORNING_MENTOR_CRON: morning mentor schedule.
- MORNING_MENTOR_TIMEZONE: default Asia/Kolkata.
- MORNING_MENTOR_TIMEZONE_OFFSET_HOURS: default 5.5.
- TIMEZONE_OFFSET_HOURS: global reporting offset.
- SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, SMTP_FROM: SMTP email config.
- RESEND_API_KEY, RESEND_FROM: Resend transactional email config.
- RATE_LIMIT_WINDOW_MS, RATE_LIMIT_MAX_REQUESTS: global rate limit.
- AUTH_RATE_LIMIT_WINDOW_MS, AUTH_RATE_LIMIT_MAX_REQUESTS: auth limiter.
- UPLOAD_RATE_LIMIT_WINDOW_MS, UPLOAD_RATE_LIMIT_MAX_REQUESTS: upload limiter.
- STATUS_RATE_LIMIT_WINDOW_MS, STATUS_RATE_LIMIT_MAX_REQUESTS: polling limiter.
- API_REQUEST_TIMEOUT_MS: API timeout.
- OCR_SERVICE_TIMEOUT_MS: OCR timeout.
- AI_SERVICE_TIMEOUT_MS: AI timeout.
- DB_OPERATION_TIMEOUT_MS: DB timeout.
- EXTERNAL_API_TIMEOUT_MS: external API timeout.
- PROCESSING_TIMEOUT_MS: overall job timeout.

Frontend Variables

- NEXT_PUBLIC_API_URL: backend API base URL.
- NEXT_PUBLIC_FIREBASE_API_KEY: Firebase web API key.
- NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN: Firebase auth domain.
- NEXT_PUBLIC_FIREBASE_PROJECT_ID: Firebase project ID.
- NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET: Firebase storage bucket.
- NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID: messaging sender ID.
- NEXT_PUBLIC_FIREBASE_APP_ID: Firebase app ID.
- NEXT_PUBLIC_FIREBASE_WEB_CLIENT_ID: Google web client ID.
- NEXT_PUBLIC_SENTRY_DSN: frontend Sentry DSN.
- SENTRY_ORG: Sentry org for source map upload.
- SENTRY_PROJECT: Sentry project for source map upload.
- SENTRY_AUTH_TOKEN: source map upload token.

Configuration Notes

- Do not read process.env directly outside backend/config/index.js except for early bootstrapping and framework-specific build config.
- JWT_SECRET and ADMIN_JWT_SECRET must be different.
- Use separate Firebase Admin and Google Vision service accounts where possible.
- In production, keep ALLOWED_ORIGINS explicit; do not wildcard all Vercel apps.
- Static-export security headers must also be set at CDN/Nginx level.
- For mobile/Capacitor, allow capacitor://localhost in CORS.

9. Security Architecture

Authentication And Authorization

- Use short-lived access JWTs.
- Use rotating refresh tokens stored hashed in MongoDB.
- Use tokenVersion to invalidate all active access tokens after sensitive events.
- Keep admin JWT secret separate from user JWT secret.
- Use separate admin middleware and route namespace.
- Always scope data queries by authenticated user ID.

Upload Security

- Enforce MIME/file-type validation.
- Enforce file size limit.
- Reject non-trade images.
- Store images in Cloudinary, not directly on API server disk.
- Delete invalid images where possible.
- Never trust AI output without validation.

API Security

- Helmet security headers.
- Production HTTPS redirect.
- CORS allowlist.
- Rate limit auth, upload, and status polling.
- Input sanitization.
- Central error handler that avoids leaking internals in production.
- Sentry scrubbing for Authorization and cookies.

Data Security

- Never store raw refresh tokens.
- Do not return password/login lock fields by default.
- Soft-delete trades for auditability.
- Clean raw OCR and AI responses after configured retention period.
- Use least-privilege keys for Cloudinary, Firebase, Google Vision, Gemini, Razorpay, and email providers.

10. Scalability Plan

First 0-1,000 Users

- Single API server.
- Embedded worker in development only; separate worker in production.
- MongoDB Atlas shared/dedicated cluster.
- Managed Redis.
- Cloudinary for images.
- Short Redis response cache for analytics.

1,000-10,000 Users

- Horizontally scale API instances.
- Scale OCR workers separately based on queue depth.
- Add worker concurrency controls by cost/provider limits.
- Add broker-specific extraction quality dashboards.
- Move expensive analytics to materialized snapshots.
- Add stricter queue dead-letter handling.

10,000+ Users

- Separate analytics read models/snapshots.
- Introduce event-driven trade write events.
- Batch AI insight generation.
- Add multi-region CDN for frontend.
- Add Mongo read replicas where appropriate.
- Add provider circuit breakers and cost controls.

11. Caching Strategy

- Cache read-heavy analytics and dashboard endpoints in Redis.
- Key by user, route, market, and query parameters.
- Do not cache auth/profile-sensitive mutation responses.
- Clear user cache after trade create, update, delete, upload confirmation, and setup/checklist writes where analytics may change.
- Keep TTLs short enough that traders trust the dashboard after updates.

12. Observability

Logs

- Use Winston for structured application logs.
- Use Morgan for HTTP request logs.
- Include route, method, status, duration, user agent, and error metadata.
- Avoid logging tokens, passwords, OTPs, payment secrets, and private keys.

Metrics To Track

- API latency by endpoint.
- OCR queue depth.
- OCR job duration.
- OCR failure rate by broker and market.
- Gemini failure/timeout rate.
- Upload-to-confirm conversion.
- Trade create/update/delete counts.
- Payment verification failures.
- Notification delivery success/failure.
- Cache hit/miss rates.

Alerts

- API 5xx spike.
- OCR queue stuck or growing.
- Redis unavailable.
- Mongo disconnected.
- Payment verification failures.
- Sentry error spike.
- AI/OCR provider timeout spike.

13. Testing Strategy

Backend

- Unit tests for metric engines, trade lifecycle, auth, payments, OCR job workflow, analytics snapshots, discipline analytics, and pattern detection.
- Integration tests for auth, admin, and trade flows.
- Security tests for IDOR, rate limits, config secrets, and token replay.
- Regression tests for Indian market parsing and upload-to-journal flows.

Frontend

- Component-level tests for key forms and dashboards.
- API hook tests with mocked responses.
- E2E smoke tests for login, upload, save trade, journal view, analytics view, and subscription gate.
- Mobile viewport checks for upload/review forms.

Critical Test Cases

- User cannot access another user's trade.
- Invalid screenshot does not create confirmed trade.
- Failed OCR job can be retried or fails cleanly.
- Trade update invalidates analytics cache.
- Payment signature verification cannot be bypassed.
- Admin token cannot access user-token-only routes and vice versa.

14. Technical Debt And Architecture Recommendations

Keep

- Separate Trade and IndianTrade models because Indian options/equity semantics differ.
- Separate admin route/auth stack.
- Repository pattern for Mongoose queries.
- Async OCR queue instead of synchronous upload processing.
- Redis user-cache invalidation after writes.

Improve

- Standardize naming: externally use Edgecipline, internally keep stratedge if needed.
- Move remaining direct DB queries from services/controllers into repositories.
- Consolidate simple Notification and richer NotificationHistory over time.
- Normalize strategy, setup, and strategyName naming across Forex and Indian flows.
- Add a formal extraction result schema instead of broad Mixed for OCRJob.extractedData.
- Add versioned analytics snapshots for faster dashboard loads at scale.
- Add API contract documentation, preferably OpenAPI.
- Add a provider abstraction for AI/OCR with explicit timeout, retry, and fallback policies.

Avoid

- Do not add broker sync in v1 architecture.
- Do not process OCR synchronously in API requests.
- Do not store raw uploaded images on local disk.
- Do not use one shared JWT secret for admin and user sessions.
- Do not allow generic wildcard production CORS.
- Do not make analytics depend on live AI calls for every page load.

15. Build Readiness Checklist

- MongoDB URI configured and reachable.
- Redis configured and reachable.
- Cloudinary keys configured.
- User and admin JWT secrets generated separately.
- Firebase web and admin config configured.
- Gemini key configured.
- Google Vision configured for best OCR quality.
- Razorpay keys configured before paid launch.
- Sentry configured for frontend and backend.
- CORS production origins explicitly set.
- API and worker deployed as separate processes.
- Data cleanup cron enabled.
- Rate limits verified.
- Security scan passes.
- Basic unit and integration test suite passes.